

Module 2

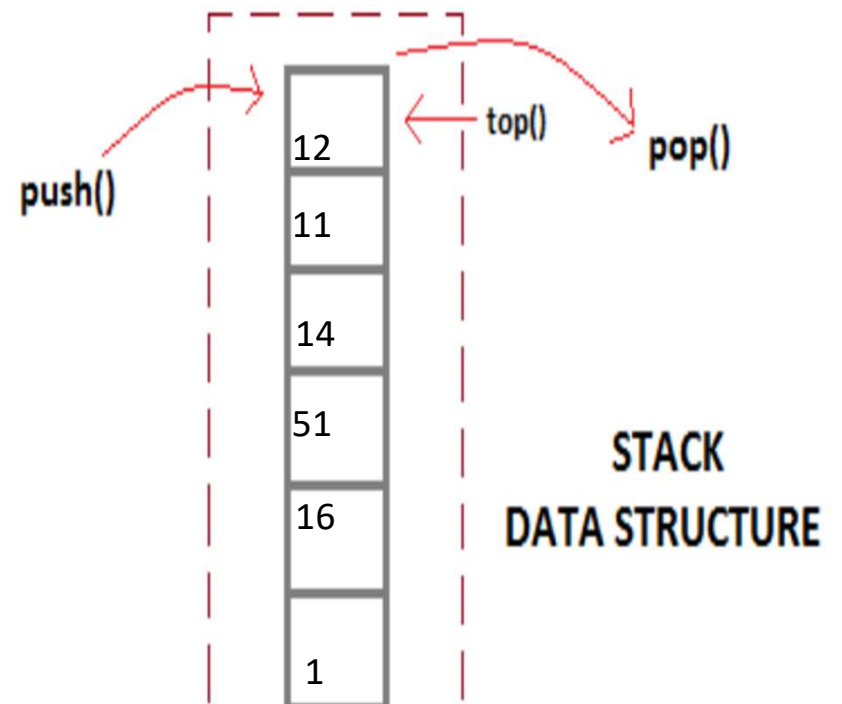
Linear Data Structures: Stacks, Queues, Representations and their Applications

Contents

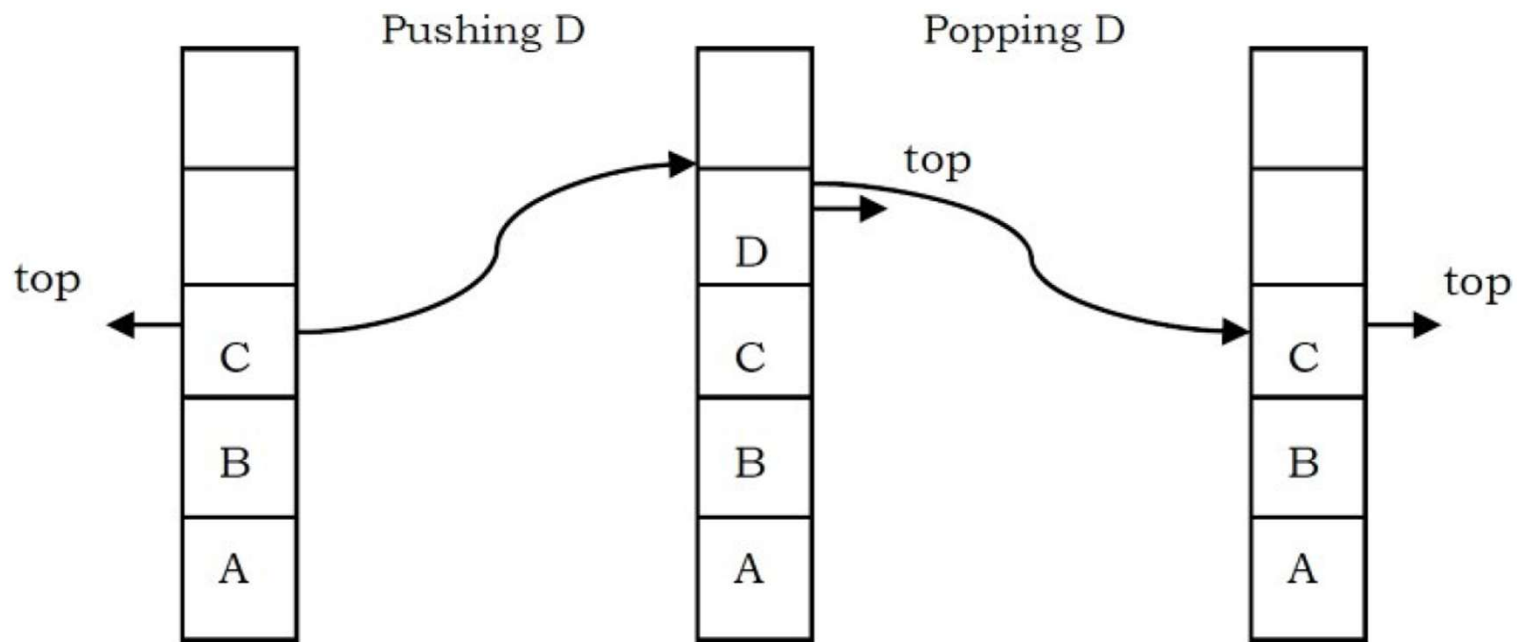
- The Stack ADT: Stack Model,
 - Implementation of Stacks:
 - Linked List Implementation of Stacks, Array Implementation of Stacks ;
 - Applications: Balancing Symbols, Postfix Expressions , Infix to Postfix Conversion, Function Calls.
- The Queue ADT: Queue Model
 - Array Implementation of Queues, Applications of Queues.

Stack

- A *stack* is an ordered list in which insertion and deletion are done at one end, called *top*.
- The last element inserted is the first one to be deleted.
- Hence, it is called the Last in First out (LIFO) or First in Last out (FILO) list.
- Stack operations:
 - **Push:** Insertion or addition of element
 - **Pop:** Deletion of an element



Stack Operations-Illustration



Stack Applications

- Balancing of symbols
- Infix-to-postfix conversion
- Evaluation of postfix expression
- Implementing function calls (including recursion)
- Page-visited history in a Web browser [Back Buttons]
- Undo sequence in a text editor
- Matching Tags in HTML and XML

Stack ADT

ADT Stack is

Data: Sequence of elements of same type

Functions:

Create(Stack): to create an empty stack and initialize the top of stack

Push(Stack, e): to insert element e into the stack

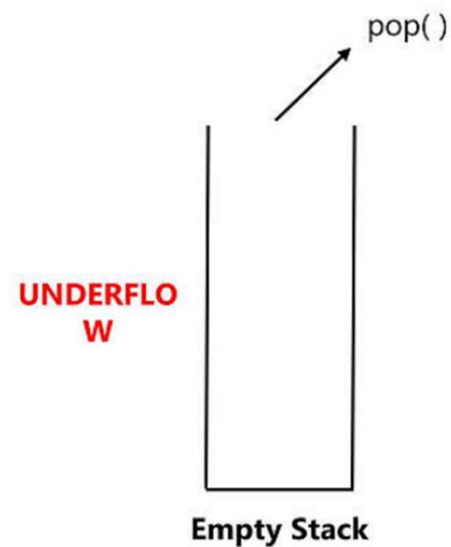
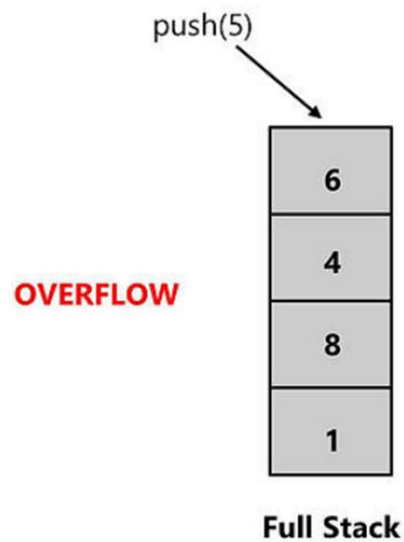
int Pop() : to pop the element from top of the stack

Auxiliary Functions:

- **int Top()**: Returns the last inserted element without removing it.
- **int Size()**: Returns the number of elements stored in the stack.
- **int IsEmptyStack()**: Indicates whether any elements are stored in the stack or not.
- **int IsFullStack()**: Indicates whether the stack is full or not.

Stack-Exceptions

- When stack is full, and we are trying to push an element, it leads to an **OVERFLOW** exception
- When stack is empty, and we are trying to pop an element, it leads to an **UNDERFLOW** exception



Stack Implementation-using Arrays

-push() function

```
void push(int stack[], int n, int top, int element)
{
    if(top==n-1) printf("Stack full: OVERFLOW!");
    else
    {
        top=top+1;//increment top
        stack[top]=element;//insert
    }
}
```

Time Complexity: $O(1)$

Stack Implementation-using Arrays

-pop() function

```
int pop(int stack[],int top, int element)
{
    if(top== -1) printf("Stack Empty: UNDERFLOW!");
    else
    {
        element=stack[top];//delete
        top=top-1;//decrement top
    }
    return element;
}
```

Time Complexity: $O(1)$

Application of Stack- Balancing Parenthesis

- Stack can be used to check whether the given expression has balanced symbols.
- This algorithm is very useful in compilers.
- Each time the parser reads one character at a time.
- If the character is an opening delimiter such as (, {, or [- then it is pushed to the stack.
- When a closing delimiter is encountered like), }, or]-the stack is popped.
- The opening and closing delimiters are then compared.
- If they match, the parsing of the string continues.
- If they do not match, the parser indicates that there is an error on the line.

Balancing Parenthesis: Pseudocode

```
Algorithm BalanceParenthesis()
{
    Create(Stack)
    //Read the expression in S. Let c=S[0];i=0;
    while(c!=End of input S){
        1)if ( c=='(' ) push(c);
        2) if(c=='') then
            if(isEmptyStack()) return Error
            else
                pop();
        3) i=i+1;
        4) c=S[i];
    }
    if(!isEmptyStack()) return Error
    else print "Expression has Balanced Parenthesis"
}
}
```

Trace the algorithm
for the expression
(A*(B-(C+D)+E))

Applications of Stack-Infix to Postfix Expression

- **Infix** expression : an expression with operators in between operands.
 - e.g. $B * C$
 - **Prefix** expression: an expression with operators before(*pre*) operands.
 - e.g. $*BC$
 - **Postfix** expression[Reverse Polish Notation]: an expression with operators after (*post*) operands.
 - e.g. $BC*$
- More Examples:

Infix	Prefix	Postfix
$A + B$	$+AB$	$AB+$
$A + B - C$	$-+ABC$	$AB+C-$
$(A + B) * C - D$	$-*+ABCD$	$AB+C*D-$

Infix to Postfix Conversion

- An infix expression can be converted by hand to postfix expression or prefix expression by decomposing the original expression into **subexpressions based on the usual rules of hierarchy, precedence and associativity.**

Token	Operator	Precedence	Associativity
() [] → .	function call array element struct or union member	17	left-to-right
-- ++	increment, decrement	16	left-to-right
-- ++ ! - - + & * sizeof	decrement, increment logical not one's complement unary minus or plus address or indirection size (in bytes)	15	right-to-left
(type)	type cast	14	right-to-left
* / %	multiplicative	13	Left-to-right
+ -	binary add or subtract	12	left-to-right
<< >>	shift	11	left-to-right
> >= < <=	relational	10	left-to-right
== !=	equality	9	left-to-right
&	bitwise and	8	left-to-right
^	bitwise exclusive or	7	left-to-right
	bitwise or	6	left-to-right
&&	logical and	5	left-to-right
	logical or	4	left-to-right
?:	conditional	3	right-to-left
= += -= /= *= %= <<= >>= &= ^=	assignment	2	right-to-left
,	comma	1	left-to-right

Infix to Postfix Conversion-Example

Let $E = a + b * c - d$

- *Equivalent postfix expression:*
 - Let $e1 = b * c \Rightarrow E = a + e1 - d$
 - $\Rightarrow$ Let $e2 = a + e1 \Rightarrow E = e2 - d$
 - $E = e2 - d$ postfix expression is $e2\ d\ -$
 - $e2 = a + e1$ postfix expression is $a\ e1\ +$
 - $e1 = b * c$ postfix expression is $bc\ *$
 - $\Rightarrow E = e2\ d\ - = a\ e1\ +\ d\ - = a\ bc\ * + d\ -$

Find its equivalent prefix expression?

Note: No parenthesis exists in the prefix expression or postfix expression

Infix to Postfix Conversion-Exercise

- *Find Equivalent postfix expressions for the following infix expressions:*
 1. $(2*3)+(4*5)$
 2. $2-3+4-5*6$
 3. $(a+b)*(d-e)+f$
 4. $A-(B+D)*(E-F)$

Infix to Postfix Conversion using Stack

Algorithm InfixtoPostfix()

{

 Create(Stack)

 //Read the expression in S. Let $c=S[0]$; $i=0$; Let $P[n]$ be an output array.

 while($c \neq$ End of input S){

 if (c is an operand) output c into P ; //Step1

 if($c=='('$) then push(c); //Step2

 else if($c==')'$) then

 pop() and output tokens into P till '(' is encountered. pop() left parenthesis. //Step3

 else {

 pop() and output tokens into P until one of lower priority than c is encountered or a left parenthesis is encountered or stack is empty //Step4

 push(c); //Step5

 }

$i=i+1$;

$c=S[i]$;

 }

 print(P);

}

Infix to postfix using Stack

Method: c=operator, operand, (,)

- if c is an operand, then output c
- **Push()** operators and '(' into stack
- if operator c is having lower priority than top character of stack, **pop()** and output the stack contents until lower priority operator is found and push c
- if ')' is found, **pop()** and o/p until '(' from stack. **pop** '('

Infix to Postfix Conversion using Stack

Algorithm InfixtoPostfix()

{

Create(Stack)

//Read the expression in S. Let $c=S[0]$; $i=0$; Let $P[n]$ be an output array.

while($c \neq$ End of input S){

if (c is an operand) then output c into P ;

if($c=='('$) then push(c);

else if($c==')'$) then

pop() and output tokens into P till '(' is encountered. pop() left parenthesis.

else {

pop() and output tokens into P until one of lower priority than c is encountered or a left parenthesis is encountered or stack is empty

push(c);

}

$i=i+1$;

$c=S[i]$;

}

}

Trace the algorithm for the expression
 $(A*B-(C+D))$

Infix to Postfix Conversion using Stack

Example: $(A * B - (C + D))$

Step	Symbol Scanned	STACK	Postfix Expression	Description
1.	(	(		Push
2.	A	(	A	
3.	*	(*	A	Push
4.	B	(*	AB	
5.	-	(-	AB*	'-' is at lower precedence than '*' pop till (and push '-'
6.	(	(-(	AB*	Push
7.	C	(-(	AB*C	
8.	+	(-(+	AB*C	Push
9.	D	(-(+	AB*CD	
10.	)	(-	AB*CD+	Pop till '('
11.	)		$AB*CD+-$	Pop till '('

Infix to Postfix Conversion-Exercise 2

- *Find Equivalent postfix expressions for the following infix expressions using stack operations:*
 1. $(2*3)+(4*5)$
 2. $2-3+4-5*6$
 3. $(a+b)*(d-e)+f$

Evaluation of Postfix Expressions

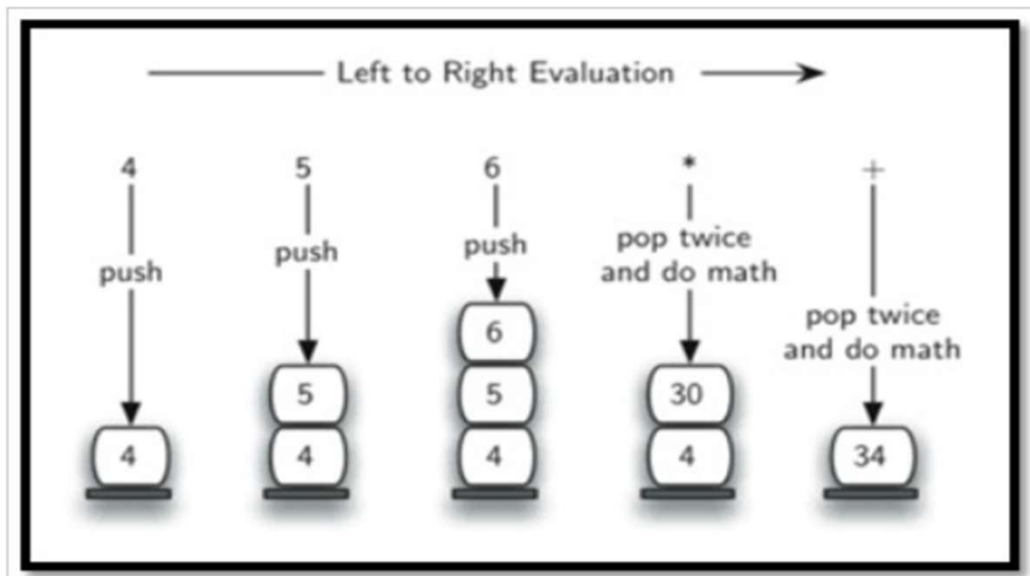
- Compiler finds it convenient to evaluate postfix expression
- Advantages
 - Elimination of parentheses which signify priority of evaluation
 - Elimination of need to observe rules of hierarchy, precedence and associativity
 - Left to right scan of the expression

Method:

- if c is an operand ,Push() operands into stack
- if c is binary operator, pop() twice and apply the operator on them

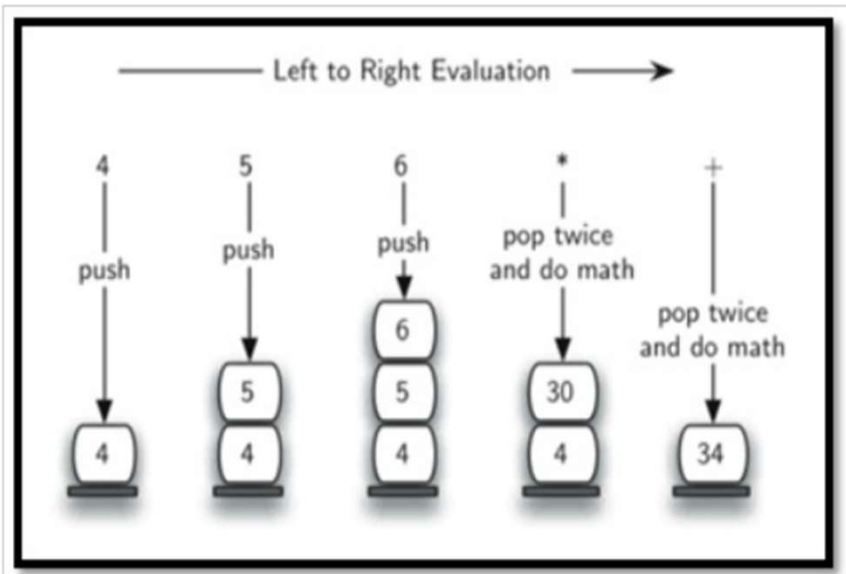
Evaluation of Postfix Expressions

Expression: 456*+



Evaluation of Postfix Expressions

Expression: 456*+



Step	Input Symbol	Operation	Stack	Calculation
1.	4	Push	4	
2.	5	Push	4,5	
3.	6	Push	4,5,6	
4.	*	Pop(2 elements) & Evaluate	4	$5*6=30$
5.		Push result(30)	4,30	
6.	+	Pop(2 elements) & Evaluate	Empty	$4+30=34$
7.		Push result(34)	34	
8.		No-more elements(pop)	Empty	34(Result)

Postfix Evaluation using Stack

Algorithm PostfixEvaluation()

{

 Create(Stack)

 //Read the expression in S and terminate it with \$. Let c=S[0];i=0;

 while(c!=End of input S){

 if (c is an operand) then push(c);

 else if(c is an operator) then

 num2=pop();num1=pop();temp=num1 c num2;push(temp)

 //pop() from the stack S, evaluate the operator and push the result into the stack S;

 else if (c=='\$') {

 result= pop() out the result from stack S;

 }

 i=i+1;

 c=S[i];

 }

 return result;

}

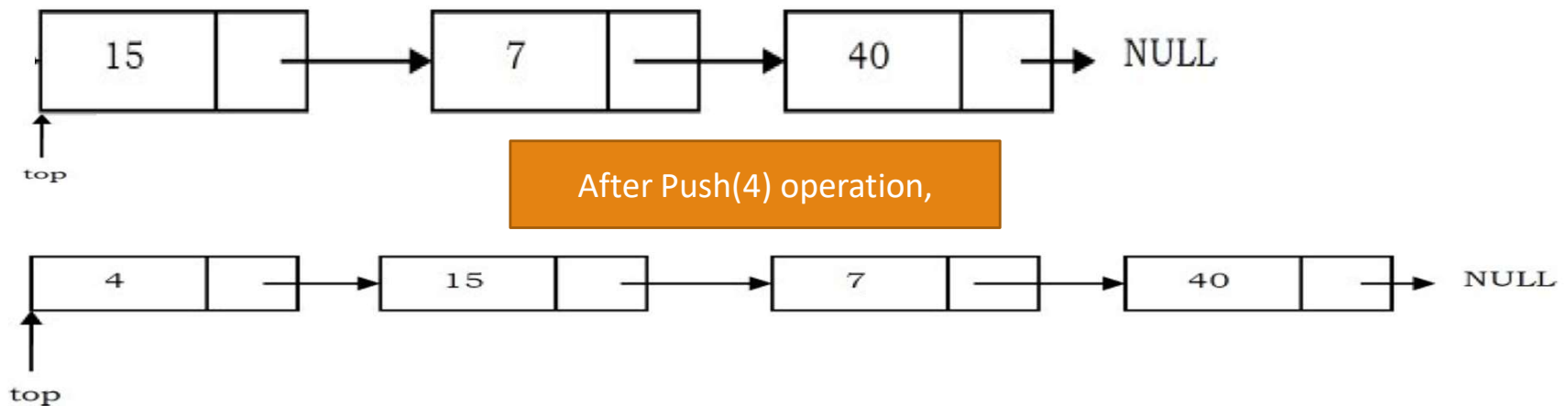
Postfix Evaluation using Stack

```
Algorithm PostfixEvaluation()  
{  
    Create(Stack)  
    //Read the expression in S and terminate it with $. Let c=S[0];i=0;  
    while(c!=End of input S){  
        if ( c is an operand) then push(c);  
        else if(c is an operator) then  
            num2=pop();num1=pop();temp=num1 c num2;push(temp)//pop() from the  
stack S, evaluate the operator and push the result into the stack S;  
            else if (c=='$') {  
                result= pop() out the result from stack S;  
            }  
            i=i+1;  
            c=S[i];  
    }  
    return result;  
}
```

Trace the algorithm for the expression
23-456*-+

Stack Implementation using Linked List Representation-Insertion

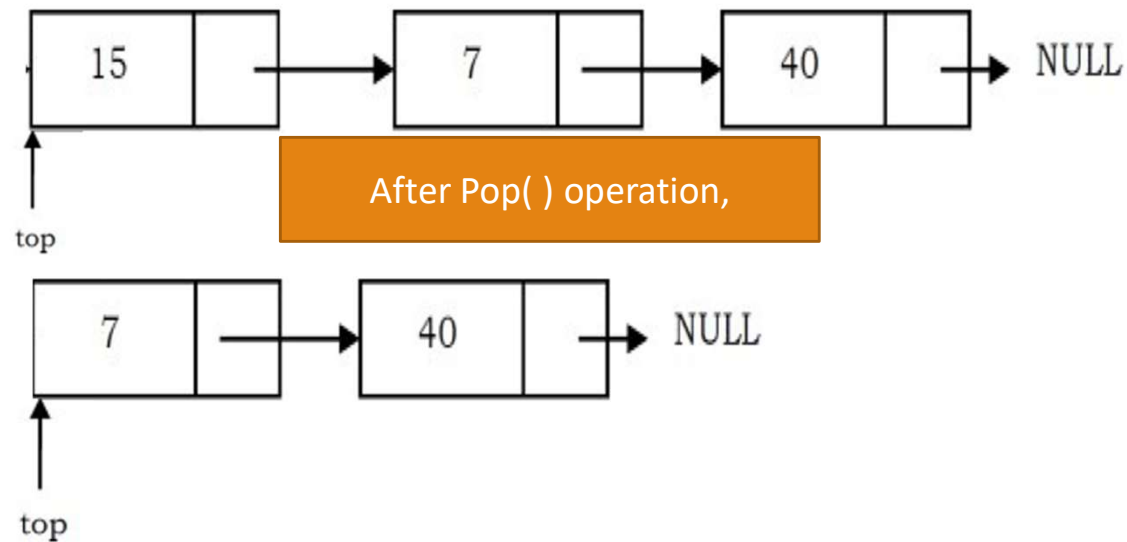
- ❑ A pointer node top is maintained for stack implementation using linked list and initialized to NULL.
- ❑ Push() operation is implemented by inserting the element at the beginning of the list.



<https://www.geeksforgeeks.org/implement-a-stack-using-singly-linked-list/>

Stack Implementation using Linked List Representation-Deletion

□ Pop() operation is implemented by deleting the node from the beginning (head/top)



Queues



Queues- Introduction

- The element at the first position is served first.
- A Queue is a FIFO (First-In, First-Out) data structure in which the element that is inserted first is the first one to be taken out.
- The elements in a queue are added at one end called the REAR and deleted from the other end called the FRONT

Queue Operations

- **Enqueue:** Element in a queue are added/inserted at one end called the REAR
- **Dequeue:** The elements in a queue are deleted from the other end called the FRONT



Queue Data Structure

Applications of Queues

Real-time:

- Luggages in a conveyer belt
- Queue of people at ticket counter
- Vehicles at toll-tax counter in highways

Computer Science related:

- In OS, for multiprogramming using CPU Scheduling
 - Disk scheduling
 - Printer Queue
 - In computer networks, routing packets
- 

Queue ADT

ADT Queue is

Data: Sequence of elements of same type

Functions:

Create(Queue): to create an empty queue and initialize FRONT and REAR(back)

Enqueue(e): to insert element e into the queue

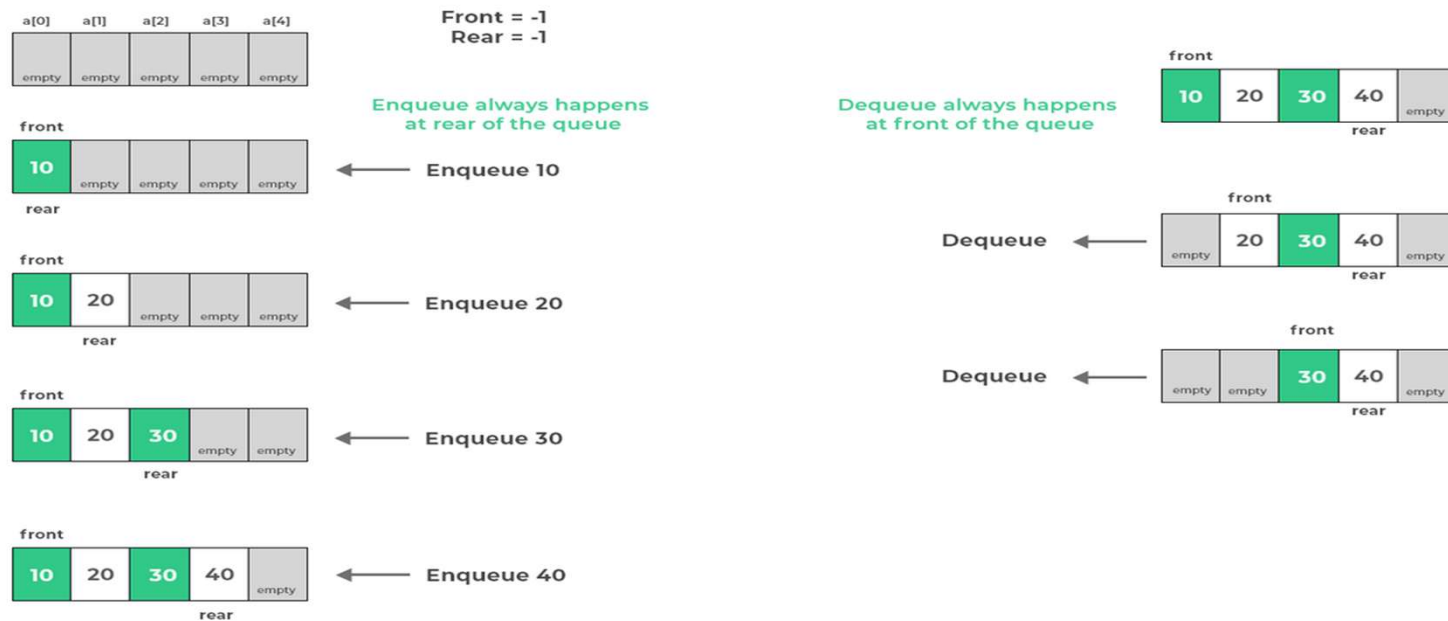
int Dequeue() : to delete the element from queue

Auxiliary Functions:

- **int Front():** Returns the element at the front node of queue without removing it.
- **int Rear():** Returns the last inserted element without removing it
- **int Size():** Returns the number of elements stored in the queue.
- **int IsEmptyQ():** Indicates whether any elements are stored in the queue or not.
- **int IsFullQ():** Indicates whether the queue is full or not.

Queue using Array Representation

Enqueue & Dequeue Operation in Queues



Exceptions in Queue Operations

- **Queue overflow:** A queue is said to be overflow when the queue is full and we are trying to insert into it.
 - Queue overflow occurs when $REAR = SIZE - 1$
- **Queue underflow:** A queue is said to be underflow when there are no elements present in the queue.
 - Queue underflow occurs when $FRONT = -1$ or $FRONT > REAR$

Array Representation (Enqueue and Dequeue)

Algorithm Enqueue(e)

```
{//Q[0:SIZE-1] is a global array
if(REAR==SIZE-1) then print "Queue OVERFLOW"
if (FRONT== -1 AND REAR== -1) then
    FRONT=REAR=0
else{
    REAR=REAR+1;
}
Q[REAR]=e;
}
```

Enqueue Algorithm

Algorithm Dequeue()

```
{
if(FRONT== -1 OR FRONT>REAR) then print "Queue UNDERFLOW"
else{
    e=Q[FRONT];
    FRONT=FRONT+1;
    return e;
}
}
```

Dequeue Algorithm

Example: Array Representation

12	9	7	18	14	36				
0	1	2	3	4	5	6	7	8	9

Figure 8.1 Queue

12	9	7	18	14	36	45			
0	1	2	3	4	5	6	7	8	9

Figure 8.2 Queue after insertion of a new element

	9	7	18	14	36	45			
0	1	2	3	4	5	6	7	8	9

Figure 8.3 Queue after deletion of an element

Discussion Point 1

Draw the queue structure in each case when the following operations are performed on an empty queue.

- (a) Enqueue A, B, C, D, E, F
- (b) Dequeue two letters
- (c) Enqueue G
- (d) Enqueue H
- (e) Dequeue four letters
- (f) Enqueue I

Discussion Point 2

Consider the queue given below which has FRONT = 1 and REAR = 5

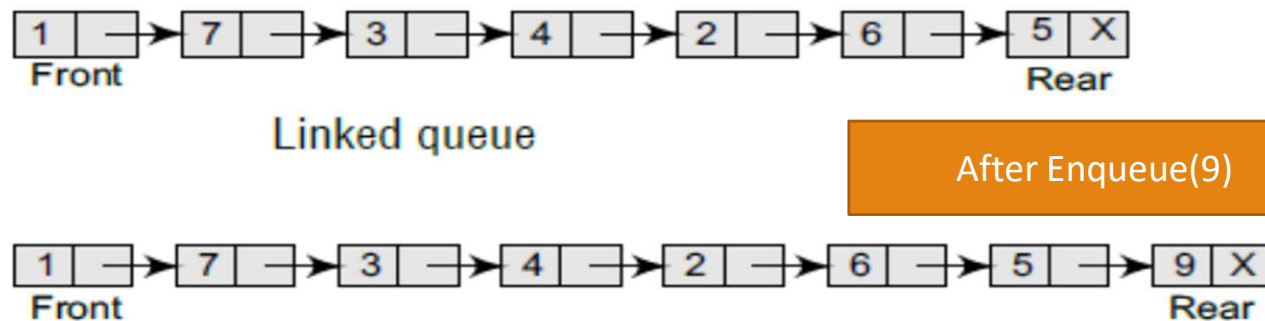


Now perform the following operations on the queue:

- (a) Add F (b) Delete two letters
- (c) Add G (d) Add H
- (e) Delete four letters (f) Add I

Queue Implementation using Linked List Representation-Insertion

- Two pointer nodes **Front and Rear** are maintained for queue implementation using linked list and initialized to NULL.
- Enqueue() operation is implemented by inserting data at the Rear end of the list.



Queue Implementation using Linked List Representation-Insertion

- ❑ Two pointer nodes **Front and Rear** are maintained for queue implementation using linked list and initialized to NULL.
- ❑ Enqueue() operation is implemented by inserting data node at the Rear end of the list and updating the rear to point to it.

- ❑ Structures in a queue:

- struct QNode{
 int data;
 struct QNode *next;
}
 - struct Queue{
 struct QNode *front,*rear;
};

```
procedure newnode(newdata)
{
    struct QNode *q=(struct QNode*)malloc(sizeof(struct QNode) );
    q->data=newdata;
    q->next=NULL;
    return q;
}
```

Enqueue() Linked List Representation

-C code

```
// The function to add a data k to q
void enqueue(struct Queue* q, int k)
{
    // Create a new LL node
    struct QNode* temp = newNode(k);

    // If queue is empty, then new node is front and rear
    // both
    if (q->rear == NULL) {
        q->front = q->rear = temp;
        return;
    }

    // Add the new node at the end of queue and change rear
    q->rear->next = temp;
    q->rear = temp;
}
```

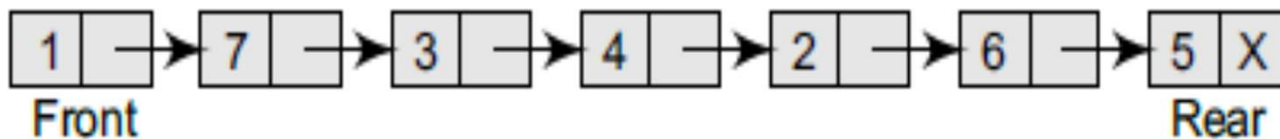
Queue Implementation: Linked List Representation-Deletion

Deque() operation is implemented by deleting node at the front end and moving front pointer to the next node



Linked queue

After Dequeue()



Linked queue after deletion of an element

DeQueue() Linked List Representation-C code

```
// Function to remove a data from given queue q
void deQueue(struct Queue* q)
{
    // If queue is empty, return NULL.
    if (q->front == NULL)
        return;

    // Store previous front and move front one node ahead
    struct QNode* temp = q->front;

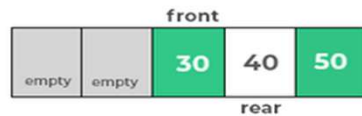
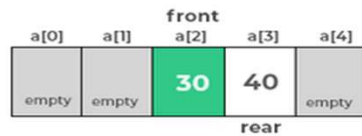
    q->front = q->front->next;

    // If front becomes NULL, then change rear also as NULL
    if (q->front == NULL)
        q->rear = NULL;

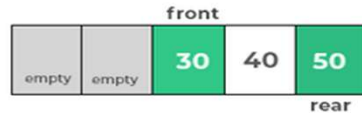
    free(temp);
}
```

Limitations with general Queues

Size = 5



← Enqueue (50)



← Enqueue (60)

Error Can't Enqueue



← Dequeue



← Dequeue



← Dequeue



← Enqueue (70)

This is the limitation in normal Queues
After multiple enqueueing and dequeuing
size of the queue gets reduced

Because the following code -

```
if (rear == SIZE-1)
    printf("Overflow condition");
```

Error Can't Enqueue

We were able to enqueue here as when the
queue becomes empty after dequeuing, it
gets reset to and both front & rear again
get value = -1

```
if (front > rear)
    front = rear = -1;
```

Will be able to enqueue now

Types of Queues

1. Circular Queue
2. Double Ended Queue (Deque)
3. Priority Queue

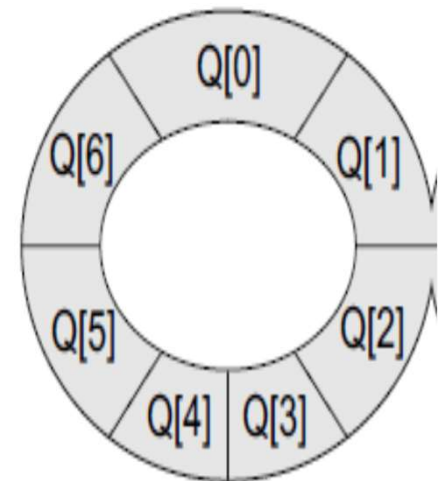
Circular Queues

54	9	7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

		7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

Circular Queues

- Major Drawback of Linear Queue: Suppose we want to insert a new element into the queue. Even though there is space available, overflow condition still exists because the condition $\text{rear} = \text{MAX} - 1$ still holds true.
- To resolve this problem, there are two options. First, to shift the elements to the left. But, this is very time consuming for large queues.
- Second option is to use the circular queues. In circular queues, first index comes right after the last index.



Insertion in Circular Queue

For insertion, 3 conditions are to be checked:

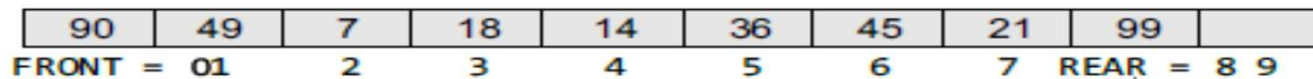
1. If **front=0 and rear=MAXSIZE-1 or $(\text{rear}+1 \% \text{MAXSIZE}=\text{front})$** then circular queue is full.

90	49	7	18	14	36	45	21	99	72
FRONT = 01	2	3	4	5	6	7	8	REAR = 9	

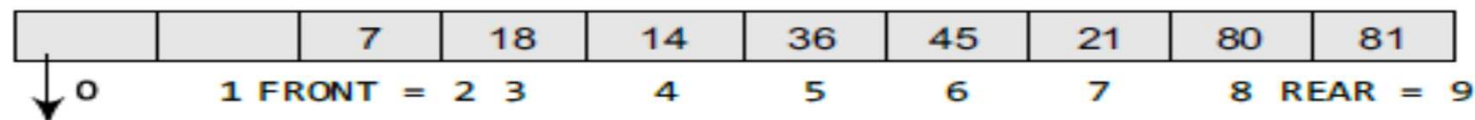
Insertion in Circular Queue

For insertion, 3 conditions are to be checked:

2. If $\text{rear} \neq \text{MAXSIZE}-1$, rear will be incremented.



3. If $\text{front} \neq 0$ and $\text{rear} = \text{MAXSIZE}-1$, it means that queue is not full. So, set $\text{rear} = 0$ and insert the element



Set REAR = 0 and insert the value here

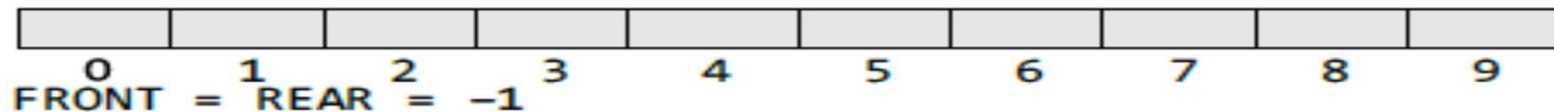
Note: For case 2,3:

$\text{rear} = (\text{rear} + 1) \% \text{MAXSIZE}$

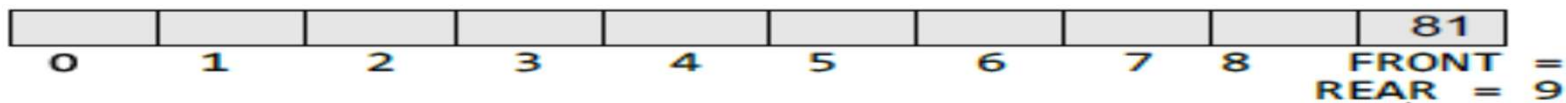
Deletion in Circular Queue

For deletion, 3 conditions are to be checked:

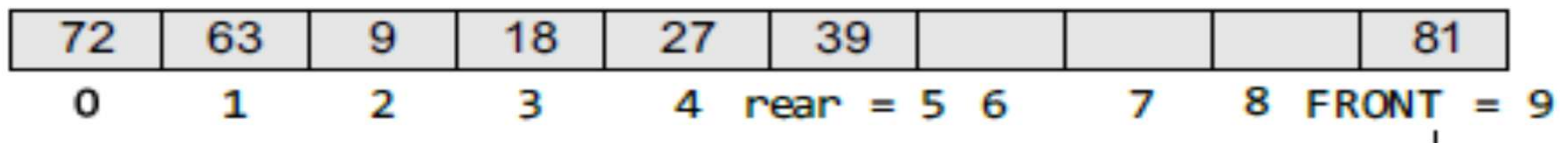
1. If $\text{front}=\text{rear}=-1$, then circular queue is empty.



2. If $\text{front}=\text{rear}$ and queue is not empty, then after deletion of the element, set $\text{front}=\text{rear}=-1$.



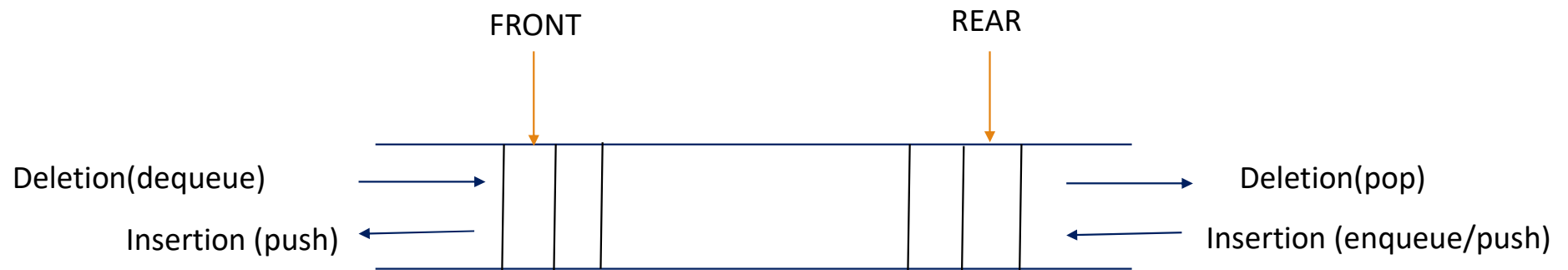
3. If $\text{front}=\text{MAXSIZE}-1$ and queue is not empty, delete the element and set $\text{front}=0$ or $\text{front}=(\text{front}+1) \bmod \text{MAXSIZE}$



Deque

- The list in which the elements can be inserted or deleted at either end
- The term deque originated from **d**ouble **e**nded **q**ueue
- It is also known as a head-tail linked list because elements can be added to or removed from either the front (head) or the back (tail) end.
- However, no element can be added and deleted from the middle.

Deque Structure



- Deque is general representation of both stack and queue.
- In the computer's memory, a deque is implemented using either a circular array or a circular doubly linked list.
- The elements in a deque extend from the LEFT end to the RIGHT end and since it is circular, $\text{Deque}[N-1]$ is followed by $\text{Deque}[0]$.

Variations of Deque

1. Input-restricted deque: insertions at one end only
2. Output-restricted deque: deletions take place at one end only

Priority Queues

A priority queue is a data structure in which each element is assigned a priority. The priority of the element will be used to determine the order in which the elements will be processed.

The general rules of processing the elements of a priority queue are:

- An element with higher priority is processed before an element with a lower priority.
- Two elements with the same priority are processed on a first-come-first-served (FCFS) basis

A priority queue can be thought of as a modified queue in which when an element has to be removed from the queue, the one with the highest-priority is retrieved first

Priority Queue- Arrays

When arrays are used to implement a priority queue, then a separate queue for each priority number is maintained.

Each of these queues will be implemented using circular arrays or circular queues.

Every individual queue will have its own FRONT and REAR pointers.

We use a two-dimensional array for this purpose where each queue will be allocated the same amount of space

Priority Queue-Arrays

- Rows denote the priority number
- Columns denote the index number

Insertion: To insert a new element with priority K in the priority queue, add the element at the rear end of row K, where K is the row number

Deletion: delete the front element of first non-empty queue

	FRONT	REAR		1	2	3	4	5
1	3	3				A		
2	1	3		B	C	D		
3	4	5					E	F
4	4	1		I			G	H

Figure 8.29 Priority queue matrix

	FRONT	REAR		1	2	3	4	5
1	3	3				A		
2	1	3		B	C	D		
3	4	1		R			E	F
4	4	1		I			G	H

Applications of Priority Queue

- Heap sort
 - Dijkstra's Algorithm
 - Prim's Algorithm
 - Data Compression-Huffman Coding technique
 - OS- Load balancing and Interrupt handling
- 